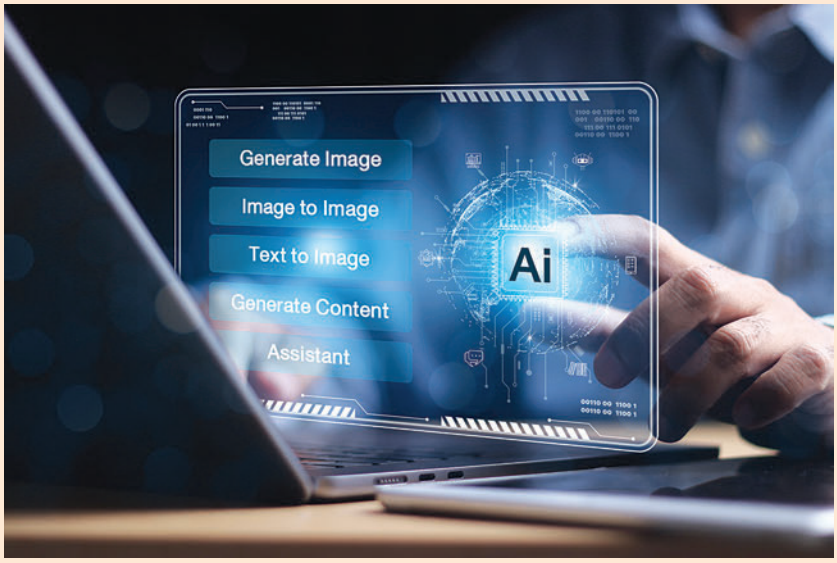
was built around real GitHub issues because real software work frequently requires changes across multiple files and functions, not a single code snippet.

For tool-using systems, evaluate the outcome, not just the conversation. Imagine a customer service AI agent that says, "Your refund has been processed." The response may be polished, but the real question is whether the refund actually exists in the transaction system. That distinction between a convincing transcript and a verified end state is where many evaluations become either useful or misleading.



Q What are the biggest mistakes companies make when evaluating an AI feature before putting it in front of real users?

A Spurthi: The first is testing what is easy to measure rather than what is important. A team may count how often an answer contains the expected phrase, even though the real requirement is whether the user's problem was resolved. Easy metrics produce neat dashboards and false confidence.

The second is using clean, ideal examples. Real users omit details, change direction, use unusual language, paste messy data, and ask for things the designers did not anticipate. An evaluation set of well-written prompts tests the product the team wishes it had, not the product people will actually use.

The third is testing the model in isolation instead of the whole product. The application also depends on company data, software tools, business rules, and access permissions, and any of those pieces can fail. The model might understand a refund request perfectly but still give the wrong answer because it retrieved an outdated policy, or identify the correct action but apply it to the wrong account.

The fourth is deciding what "good enough" means after seeing the results. A feature underperforms, so the team lowers the bar to launch anyway. Instead, decide in advance what the system must achieve, which failures are unacceptable, and when a human must step in, matched to the risk involved. A weak headline suggestion may be tolerable; an incorrect payment or medical recommendation may not be.

Q Can an AI system look impressive in a demo or benchmark and still be bad in production? What causes that gap?

A Spurthi: Absolutely. A demo is usually a best-case path. The prompt is known, the data is prepared, and the example is hand-picked to show the product at its best. Production is full of incomplete information, unusual requests and changing data.

Take the refund agent again. In a demo, it gets a clear request and does everything. In production, the customer may have two accounts, the policy document may be outdated, a tool call may time out, or a malicious instruction may be hidden in retrieved content. The model can be impressive while the end-to-end system remains fragile.



Because generative systems are non-deterministic, succeeding once does not mean they will succeed consistently. Recent NIST work on deployed AI systems emphasizes that controlled pre-release evaluation cannot fully reproduce real-world dynamics and should be complemented by repeated post-deployment testing and monitoring.

Security widens that gap. An AI system can be manipulated by instructions hidden in a user message, document, or webpage, known as prompt injection. The damage is much greater if the agent has excessive agency, meaning more access or authority than it needs. A support agent should not be able to issue a large refund or change sensitive account details without human approval. The right question is not only, "Can the AI complete the task?" It is also, "What is it allowed to do if it misunderstands the task?"

Q For someone building an AI app through APIs, no-code tools or vibe coding, what is the simplest practical way to test reliability?

A Spurthi: Start with a test set drawn from real use. 20-50 realistic cases are a useful starting point for an early-stage system, especially if they include known